LOW-LEVEL DESIGN

Customer Support RAG Assistant

Document Purpose

This document provides a detailed low-level design reference for the Customer Support RAG Assistant, covering module architecture, data structures, LangGraph workflow design, conditional routing logic, HITL integration, and error handling strategies.

5 Modules	4 Graph Nodes	3 Error Handlers
Core Python Components	LangGraph State Machine	Fault Tolerance Layers

01 Module-Level Design

The system is decomposed into five discrete Python modules, each with a clearly bounded responsibility. This separation of concerns ensures that individual components can be tested, replaced, or scaled independently without requiring modifications to adjacent modules.

Module	File	Responsibility
Document Processing	`src/ingestion.py`	Reads source data files, splits content into fixed-size chunks, and manages batched submission to the embedding pipeline. Handles retry logic and progress tracking.
Embedding Module	`src/embeddings.py`	Interfaces with langchain-huggingface to generate dense vector representations from text chunks using a locally hosted transformer model.
Vector Storage	`index/ (ChromaDB)`	Persists and indexes all embeddings to disk using ChromaDB. Supports similarity search queries with configurable top-K and minimum score thresholds.
Graph Execution	`src/graph_logic.py`	Defines the LangGraph state machine including all node functions, conditional edge logic, and state transition rules governing the query lifecycle.
API Interface	`api.py`	Exposes RESTful FastAPI endpoints consumed by the React frontend. Manages request validation, error handling, and response serialization.

02 Data Structures

2.1 AgentState — Core State Object

The AgentState TypedDict is the single source of truth passed between all LangGraph nodes. Every node reads from and writes to this shared object, enabling the graph to maintain full context across the query lifecycle without external side effects.

```
class AgentState(TypedDict):
    query:                str     # Raw user query string
    context:              str     # Retrieved document chunks (joined)
    response:             str     # Final LLM-generated response
    escalated:            bool    # True if routed to human escalation
    needs_confirmation:   bool    # True if UI should render HITL buttons
    history:              List[str]  # Ordered list of prior turns
```

Field-level design notes:

- query — Immutable after initial assignment; all nodes reference this field but none may overwrite it, preserving the original user intent throughout the graph.
- context — Populated by the retrieve node as a concatenated string of top-K document chunks. Used exclusively by the generate node.
- response — Written only by generate or suggest_escalate. The API layer reads this field for the final response payload.
- escalated — Boolean flag that persists across the session to prevent re-routing an already-escalated query back through the standard pipeline.
- needs_confirmation — Consumed by the React frontend to conditionally render human handoff action buttons without requiring a separate API call.
- history — A mutable list appended to after each complete query-response cycle, enabling future multi-turn conversational memory support.

2.2 API Schema

The FastAPI layer exposes a single primary endpoint with the following typed request and response schemas:

Request Body (POST /query)	Response Body

`{ "query": string }`
 	
query — Required. The user's natural language support question.
	`{ "response": string,`
	`"escalated": bool,`
	`"needs_confirmation": bool,`
	`"context": string }`

03 Workflow Design (LangGraph)

The query lifecycle is modeled as a LangGraph Stateful Graph — a directed state machine where each node is a Python function that accepts and returns an AgentState object. Conditional edges determine routing at decision points, enabling branching and early termination without imperative control flow.

3.1 Node Definitions

`retrieve` — Accepts the raw query from AgentState. Generates an embedding vector for the query and performs a top-K similarity search against the ChromaDB index. Populates the context field with concatenated chunk text. Returns the updated state.

`router` — Passes the query and retrieved context to a lightweight LLM routing prompt. Evaluates intent classification and context sufficiency. Sets escalated and needs_confirmation flags based on a confidence threshold of 0.7. Does not modify query or context.

`generate` — Invokes the generation LLM with a detailed prompt that includes the query, context, and response formatting instructions. Writes the final answer to the response field. Only executed when the router has confirmed confidence >= 0.7.

`suggest_escalate` — Constructs a templated escalation offer message and writes it to the response field. Sets needs_confirmation to True so the frontend renders actionable human handoff buttons. Executed when router confidence falls below threshold.

3.2 State Transition Diagram

The diagram below shows the conditional routing paths through the LangGraph state machine. The confidence threshold of 0.7 governs the branching decision at the Router node.

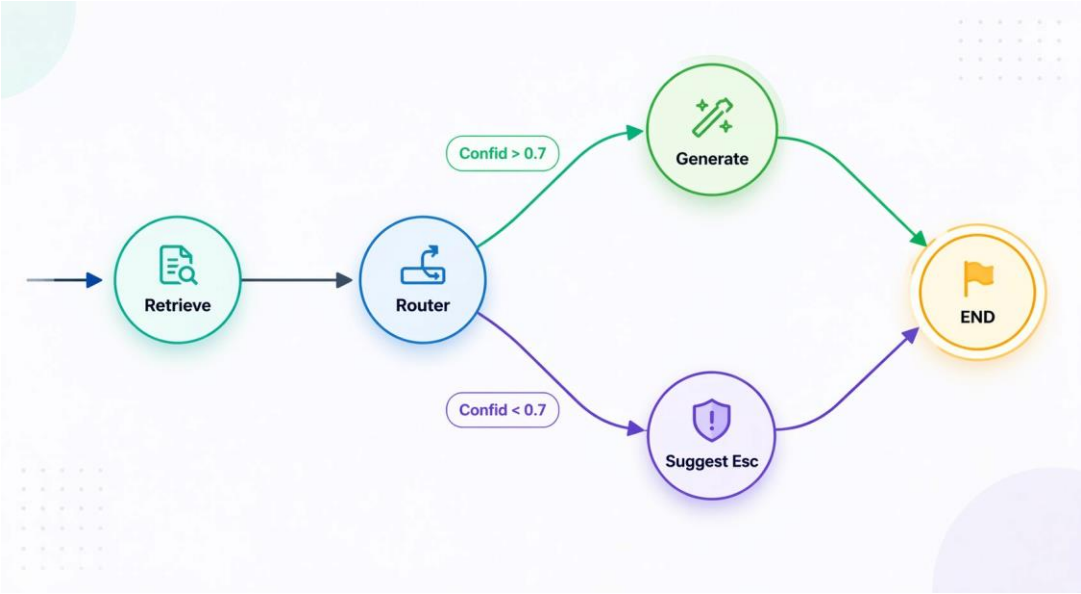

Figure 1: LangGraph State Transition Diagram — Conditional Routing at the Router Node

3.3 Edge Conditions

```
# Conditional edge registered on the Router node
def routing_condition(state: AgentState) -> str:
    if state["escalated"]:
        return "suggest_escalate"
    if state["needs_confirmation"]:
        return "suggest_escalate"
    return "generate"

graph.add_conditional_edges(
    "router",
    routing_condition,
    { "generate": generate_node, "suggest_escalate": escalate_node }
)
```

04 Conditional Routing Logic

The Router node applies a multi-criteria decision function to classify each query into one of two routing outcomes. The criteria are evaluated in priority order; the first matching condition determines the route.

4.1 Criteria for ANSWER Route (Confidence >= 0.7)

- The retrieval step returns at least one chunk with a cosine similarity score above the configured minimum threshold (default: 0.65).
- The LLM routing prompt classifies the query intent as a recognized support category (refund, order status, shipping, returns, or policy inquiry).
- The combined token length of retrieved chunks fits within the generation prompt context window without truncation.
- The query does not contain any explicit escalation trigger keywords (see Section 4.2).

4.2 Criteria for ESCALATE Route (Confidence < 0.7)

- Explicit keyword match — the query contains phrases such as 'talk to human', 'speak to agent', 'I want a person', or 'human support'.
- Low retrieval quality — all returned chunks have similarity scores below the minimum threshold, indicating the query is outside the knowledge base scope.
- LLM uncertainty signal — the routing LLM's internal reasoning includes uncertainty markers such as 'I don't know', 'I'm not sure', or 'insufficient information'.
- Sensitive query classification — the router identifies the query as involving a legal dispute, fraud allegation, or complaint against staff, which are mandatorily escalated regardless of retrieval quality.

4.3 Confidence Score Composition

The 0.7 threshold is a composite score computed from three weighted sub-signals:

Signal	Weight	Description
Retrieval Score	**50%**	Average cosine similarity of top-K retrieved chunks
Intent Match	**30%**	Binary classification: recognized intent = 1.0, unknown = 0.0
LLM Confidence	**20%**	Self-reported confidence from the routing prompt output

05 HITL (Human-in-the-Loop) Design

The Human-in-the-Loop subsystem provides a controlled escalation pathway for queries that exceed the system's automated resolution capability. It is designed to maintain a seamless user experience by keeping the interaction within the same UI surface rather than redirecting to a separate support channel.

5.1 Trigger Mechanism

HITL is triggered exclusively by the Router node returning the suggest_escalate routing outcome. The trigger is non-reversible within a session: once escalated is set to True in AgentState, all subsequent queries in the same session are routed directly to suggest_escalate without re-evaluating retrieval or confidence.

5.2 Frontend UI State

When the API response includes needs_confirmation: true, the React frontend transitions from the standard response display mode to an interactive escalation confirmation state:

- Two action buttons are rendered: Confirm — Connect me to an agent and Cancel — Try another question.
- The standard text input is temporarily disabled to prevent new queries from being submitted while the confirmation is pending.
- The escalation offer message from the response field is displayed as a styled card with distinct visual treatment to differentiate it from standard AI responses.

5.3 Backend Escalation Flow

Upon receiving a user confirmation click, the frontend submits a virtual query string ('__HUMAN_CONFIRMED__') to the POST /query endpoint. The API layer detects this sentinel value and bypasses the full LangGraph execution, instead invoking a dedicated escalation handler that:

- Generates a professional, empathetic handoff message acknowledging the user's request.
- Logs the escalation event with the full query history to the support ticket system.
- Returns a final response with escalated: true and needs_confirmation: false, signaling the frontend to return to standard display mode.

5.4 Simulated Agent Response

In the current implementation, the 'human agent' response is simulated using a specialized generation prompt that adopts a human support agent persona with formal tone, empathy markers, and explicit next-step instructions. This enables end-to-end testing of the escalation flow without requiring live agent integration.

Production deployment will replace this simulated response with a webhook to an external CRM or ticketing system (e.g., Zendesk, Freshdesk), with the LangGraph session state serialized and attached to the created ticket for agent context.

06 Error Handling

The system implements three layers of error handling, each targeting a distinct failure mode. All error handlers return structured API responses with consistent schema, ensuring the frontend always receives a predictable payload regardless of the failure source.

6.1 Missing Vector Index

On startup, api.py checks for the presence of the index/ directory and validates that it contains a non-empty ChromaDB collection. If the check fails:

- If an ingestible data source is available (e.g., the source CSV is present), IngestionPipeline.run() is triggered automatically to rebuild the index before accepting any queries.
- If no data source is available, the API returns a 503 Service Unavailable response with a descriptive error body and logs a CRITICAL-level event to the application logger.
- A startup health check endpoint (GET /health) exposes index status so external monitoring systems can detect and alert on this condition without requiring a failed query.

6.2 LLM API Failure

All LLM calls within graph_logic.py are wrapped in try-except blocks that catch both Groq API errors and general network exceptions:

```
try:
    response = llm.invoke(prompt)
except GroqAPIError as e:
    logger.error(f"LLM API failure: {e}")
    state['response'] = (
        "Our AI assistant is temporarily unavailable. "
        "Please try again in a moment, or contact support directly."
    )
    state['escalated'] = True
    return state
```

This ensures the user always receives a coherent response rather than an unhandled exception, and the automatic escalation flag ensures the frontend offers human assistance as a fallback.

6.3 Port Conflict Resolution

The application implements a port availability check on startup using Python's socket library. If the configured port (default 8080) is occupied, the check logic:

- Attempts the next three consecutive ports (8081, 8082, 8083) in sequence.
- Logs a WARNING-level event indicating the port fallback with the actual bound port.
- Writes the active port to a runtime configuration file (runtime.env) which the frontend reads at startup to configure its API base URL dynamically.

This eliminates the 502 Bad Gateway errors observed during development when port 8000 was in use, and ensures zero-configuration startup in shared development environments.